

Pseudo Classes:

A [CSS](#) **pseudo-class** is a keyword added to a selector that lets you style a specific state of the selected element.

Characteristics of Pseudo Classes:

Syntax: Pseudo-classes are declared using a single colon (:) followed by the name of the pseudo-class, appended to a selector.

```
selector:pseudo-class {  
  property: value;  
}
```

Purpose: They target elements only when they are in a specific state, such as when they are being hovered over, when they have been visited, or when they are the first child of their parent.

1. User Action (Dynamic) Pseudo-Classes

These respond to user interaction.

Pseudo-Class	Description	Example
<code>:hover</code>	When user places the mouse pointer over an element	<code>a:hover { color: red; }</code>
<code>:active</code>	When an element is being clicked	<code>button:active { transform: scale(0.9); }</code>
<code>:focus</code>	When an element (like input) is focused	<code>input:focus { border-color: blue; }</code>
<code>:visited</code>	Applies to links that have been visited	<code>a:visited { color: purple; }</code>
<code>:link</code>	Applies to links that haven't been visited	<code>a:link { color: blue; }</code>


```
input[type="text"]:focus {
    border-color: dodgerblue;
    box-shadow: 0 0 5px dodgerblue;
}

button {
    padding: 10px 20px;
    background-color: #28a745;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    transition: transform 0.2s, background-color 0.3s;
}

button:hover {
    background-color: #218838;
}

button:active {
    transform: scale(0.95);
    background-color: #1e7e34;
}
</style>
</head>

<body>

    <h2>User Action Pseudo-Classes </h2>
    <p>
        <a href="https://www.google.com" target="_blank">Visit Google</a>
    </p>
    <p>
        <label>Enter your name:</label><br>
        <input type="text" placeholder="Click or focus here">
    </p>
    <p>
        <button>Click Me</button>
    </p>
</body>
</html>
```

2. Structural Pseudo-Classes

These select elements based on their position in the DOM tree.

Pseudo-Class	Description	Example
<code>:first-child</code>	Selects the first child of a parent	<code>p:first-child { color: red; }</code>
<code>:last-child</code>	Selects the last child of a parent	<code>p:last-child { color: blue; }</code>
<code>:nth-child(n)</code>	Selects elements based on position (1-based index)	<code>li:nth-child(2) { color: green; }</code>
<code>:nth-last-child(n)</code>	Same as above, but counting from the end	<code>li:nth-last-child(1)</code>
<code>:only-child</code>	Selects elements that are the only child of their parent	<code>p:only-child { font-style: italic; }</code>
<code>:first-of-type</code>	Selects the first element of a specific type	<code>p:first-of-type { color: orange; }</code>
<code>:last-of-type</code>	Selects the last element of a specific type	<code>p:last-of-type { color: pink; }</code>
<code>:nth-of-type(n)</code>	Selects nth element of a given type	<code>p:nth-of-type(2)</code>
<code>:empty</code>	Selects elements with no children	<code>div:empty { display: none; }</code>

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Structural Pseudo-Class Example</title>
```

```
<style>
  ul {
    list-style: none;
    padding: 0;
    width: 200px;
    font-family: Arial, sans-serif;
  }

  ul li {
    margin: 5px 0;
    padding: 10px;
    border-radius: 4px;
    text-align: center;
  }

  ul li:nth-child(odd) {
    background-color: #bbdefb;
  }

  ul li:nth-child(even) {
    background-color: #e3f2fd;
  }

  ul li:first-child {
    background-color: #b9f6ca;
    font-weight: bold;
  }

  ul li:last-child {
    background-color: #ff8a80;
    font-weight: bold;
  }
</style>
</head>
<body>

<h3>Structural Pseudo-Classes with List</h3>

<ul>
  <li>Item 1 (first child)</li>
  <li>Item 2</li>
  <li>Item 3</li>
  <li>Item 4</li>
  <li>Item 5 (last child)</li>
</ul>
```

```
</body>
</html>
```

3. Form Pseudo-Classes

These target elements based on their state in a form or other user interface context.

Pseudo-Classes	Description	Example
<code>:checked</code>	Selects radio buttons or checkboxes that are checked.	<code>input:checked + label { color: green; }</code>
<code>:enabled</code>	Selects user interface elements (like <code><input></code> , <code><button></code>) that are enabled.	<code>input:enabled { opacity: 1; }</code>
<code>:disabled</code>	Selects user interface elements that are disabled.	<code>input:disabled { opacity: 0.5; }</code>
<code>:required</code>	Selects form elements that have the <code>required</code> attribute.	<code>input:required { border-left: 5px solid red; }</code>

:valid, :invalid	Selects form elements whose content is valid or invalid based on the input type (e.g., email format).	invalid valid
-----------------------------------	---	---

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Form Pseudo-Class Example</title>
<style>
  body {
    font-family: Arial, sans-serif;
    background: #f5f5f5;
    padding: 30px;
  }

  form {
    width: 300px;
    margin: auto;
    background: white;
    padding: 20px;
    border-radius: 8px;
  }

  label {
    display: block;
    margin-top: 10px;
  }

  input {
    width: 100%;
    padding: 8px;
    margin-top: 4px;
    border: 2px solid #ccc;
    border-radius: 4px;
    outline: none;
  }

  input:focus {
    border-color: dodgerblue;
  }

```

```

}

input:required {
  border-left: 4px solid red;
}

input:valid {
  border-color: green;
}

input:invalid {
  border-color: red;
}

input[type="checkbox"]:checked + label {
  color: green;
  font-weight: bold;
}

button {
  margin-top: 15px;
  width: 100%;
  padding: 8px;
  background: #007bff;
  color: white;
  border: none;
  border-radius: 4px;
}
</style>
</head>
<body>

<form>
  <label for="name">Name (required):</label>
  <input type="text" id="name" name="name" required>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" placeholder="example@mail.com">

  <input type="checkbox" id="agree">
  <label for="agree">I agree</label>

  <button type="submit">Submit</button>
</form>

```



```
</body>
</html>
```

4. Negation and Logical Pseudo-Classes

Pseudo-Class	Description	Example
<code>:not(selector)</code>	Excludes elements matching the selector	<code>p:not(.highlight) { color: gray; }</code>
<code>:is(selector)</code>	Matches if element fits <i>any</i> of listed selectors	<code>:is(h1, h2, h3) { font-weight: bold; }</code>
<code>:where(selector)</code>	Similar to <code>:is()</code> , but has no specificity	<code>:where(section, article) { margin: 20px; }</code>
<code>:has(selector)</code>	Selects element containing another matching element	<code>div:has(img) { border: 1px solid #333; }</code>

```
<!DOCTYPE html>
<html lang="en">
<head>
<style>
  div {
    width: 200px;
    padding: 10px;
    margin: 8px;
    text-align: center;
    border: 2px solid #999;
  }

  div:not(.special) {
    background-color: red;
  }
</style>
</head>
```

```
<body>

<h2>:not() Pseudo-Class Example</h2>

<div>Box 1</div>
<div class="special">Box 2 (special)</div>
<div>Box 3</div>

</body>
</html>
```

Tooltips:

A `tooltip` is a contextual text bubble that displays a description for an element that appears on pointer hover or keyboard focus

```
<!DOCTYPE html>
<html>
<style>
    .tooltip {
        position: relative;
        display: inline-block;
        border-bottom: 1px dotted black;
        cursor: pointer;
    }

    .tooltiptext {
        visibility: hidden;
        width: 130px;
        background-color: black;
        color: #ffffff;
        text-align: center;
        border-radius: 6px;
        padding: 5px 0;
        position: absolute;
        z-index: 1;
        bottom: 100%;
        left: 65%;
        margin-left: -65px;
    }

    .tooltip:hover .tooltiptext {
        visibility: visible;
    }
</style>

<div class="tooltip">
    Hover me
</div>
```

```

    }
</style>

<body>

    <h2>Top-aligned Tooltip</h2>
    <p>Move the mouse over the text below:</p>

    <div class="tooltip">Hover over me
        <span class="tooltiptext">Some tooltip text</span>
    </div>

</body>

</html>

```

Images:

In **CSS**, an **image** is any **graphic resource** (like a photo, icon, or pattern) that is used as a **style** — not as page content.

It is not placed in the HTML `<img>` tag but is **applied through CSS properties** to decorate or enhance elements.

```

<!DOCTYPE html>
<html>

<head>
    <style>
        img {
            display: block;
            margin: auto;
            width: 20%;
        }
    </style>
</head>

<body>
    <h2>Center an Image Horizontally</h2>

```

```

    <p>To center an image horizontally, set margin to auto, convert it into a
    block element, and define a width:</p>
    
    <img src="" alt="" srcset="">
</body>

</html>

```

Image Filter :

The **filter** [CSS](#) property applies graphical effects like blur or color shift to an element. Filters are commonly used to adjust the rendering of images, backgrounds, and borders

Filter	Description	Example
blur()	Adds a Gaussian blur	<code>filter: blur(5px);</code>
brightness()	Adjusts brightness	<code>filter: brightness(150%);</code>
contrast()	Increases or decreases contrast	<code>filter: contrast(120%);</code>
grayscale()	Converts to black & white	<code>filter: grayscale(100%);</code>
invert()	Inverts colors	<code>filter: invert(100%);</code>
opacity()	Adjusts transparency	<code>filter: opacity(70%);</code>
saturate()	Adjusts color saturation	<code>filter: saturate(200%);</code>
sepia()	Gives an old photo (brownish) effect	<code>filter: sepia(80%);</code>
drop-shadow()	Adds shadow similar to <code>box-shadow</code>	<code>filter: drop-shadow(10px 10px 5px gray);</code>

Background-Image:

Property	Possible Values	Description
background-image	<code>url("image.jpg")</code> , <code>none</code> , <code>linear-gradient()</code> , <code>radial-gradient()</code>	Sets one or more background images for an element.
background-repeat	<code>repeat</code> , <code>no-repeat</code> , <code>repeat-x</code> , <code>repeat-y</code> , <code>space</code> , <code>round</code>	Defines how the background image repeats horizontally or vertically.
background-position	<code>left</code> , <code>right</code> , <code>top</code> , <code>bottom</code> , <code>center</code> , or <code>x% y%</code>	Sets the starting position of the background image.
background-size	<code>auto</code> , <code>cover</code> , <code>contain</code> , <code><length></code> , <code><percentage></code>	Specifies how the background image should be scaled.
background-attachment	<code>scroll</code> , <code>fixed</code> , <code>local</code>	Determines whether the background image scrolls with the page or stays fixed.
background-origin	<code>border-box</code> , <code>padding-box</code> , <code>content-box</code>	Defines where the background image starts painting inside the element box.
background-clip	<code>border-box</code> , <code>padding-box</code> , <code>content-box</code> , <code>text</code>	Specifies how far the background should extend (useful with <code>background-clip: text</code>).
background-color	Any color value (<code>red</code> , <code>#ff0000</code> , <code>rgb()</code> , <code>transparent</code>)	Sets the background color behind the image.
background-blend-mode	<code>multiply</code> , <code>screen</code> , <code>overlay</code> , <code>darken</code> , <code>lighten</code> , etc.	Blends the background image with the background color.
background	Shorthand property	Allows setting multiple background properties in a single line.

```

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Background Fixed Example</title>
  <style>
    body {
      margin: 0;
      height: 200vh;
      background-image: url("https://picsum.photos/1200/800");
      background-repeat: no-repeat;
      /* background-position: center; */
      background-size: cover;
      background-attachment: fixed;
    }
  </style>
</head>

<body>
</body>

</html>

```

Buttons:

The `<button>` [HTML](#) element is an interactive element activated by a user with a mouse, keyboard, finger, voice command, or other assistive technology. Once activated, it then performs an action, such as submitting a [form](#) or opening a dialog.

```

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Buttons</title>
  <style>
    button,
    input[type=button] {

```

```
        border: none;
        padding: 10px 20px;
        color: white;
        font-size: 16px;
        margin: 5px;
        cursor: pointer;
        border-radius: 5px;
    }

    .primary {
        background: #1500ff;
    }

    .danger {
        background: #f44336;
    }

    .info {
        background: #2196F3;
    }

    .warning {
        background: #ff9800;
    }

    .success {
        background-color: #28A745;
    }

    button:hover {
        opacity: 0.8;
    }
</style>
</head>

<body>

    <button class="primary">Primary</button>
    <button class="danger">Danger</button>
    <!-- <button class="info">Info</button> -->
    <!-- <button class="warning">Warning</button> -->
    <input type="button" value="Info" class="info">
    <input type="button" value="Warning" class="warning">
    <input type="button" value="Success" class="success">
```

```
</body>
```

```
</html>
```

User Interface:

User Interface (UI) properties are used to manage how a web page's elements **interact with the user**.

These properties define how users can **click, focus, select, resize, or navigate** elements visually.

CSS User Interface Properties

They are used to:

- Control **user interactivity** (e.g., can the user select text?)
- Change **cursor style**
- Define **focus outlines** (for accessibility)
- Control **appearance** of form controls
- Handle **resize, overflow, and scrolling**
- Provide **visual feedback** on hover, focus, or disabled states

Property	Function	Example
cursor	Sets the style of the mouse pointer	<code>cursor: pointer;</code>
resize	Lets users resize elements like <code><textarea></code>	<code>resize: both;</code>
outline	Draws a border around focused elements	<code>outline: 2px solid blue;</code>


```
    textarea {
      resize: both;
      width: 200px;
      height: 100px;
    }

    input {
      caret-color: crimson;
      outline: none;
      border: 2px solid #ccc;
      padding: 6px;
    }

    input:focus {
      border-color: dodgerblue;
    }

    .block {
      user-select: none;
      background: lightgray;
      padding: 10px;
    }
  </style>
</head>

<body>

  <h3>CSS User Interface Properties Example</h3>

  <input type="text" placeholder="Click or focus here">
  <br><br>

  <textarea>Try resizing me</textarea>
  <br><br>

  <button>Enabled Button</button>
  <button disabled>Disabled Button</button>
  <br><br>

  <div class="block">Text selection is disabled here</div>
</body>
</html>
```

Box-Sizing :

The **box-sizing** property in CSS defines how the total width and height of an element are calculated — whether they include the element's **padding** and **border** or not. By default, when you set the width and height of an element, **CSS does not include padding or border** in those measurements — which can cause layout issues.

Syntax : box-sizing: content-box | border-box | inherit;

Value	Description
content-box (default)	Width and height include only the content area . Padding and border are added outside of it.
border-box	Width and height include content + padding + border . This makes it easier to size elements precisely.
inherit	Inherits the box-sizing value from its parent element.

Example :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    div{
      border : 1px solid;
      width: 500px;
      margin: 5px;
      padding: 4px;
      font-size : 25px;
    }

    .b-box{
      box-sizing: border-box;
    }

    .c-box{
      box-sizing: content-box;
    }
  </style>
```

```

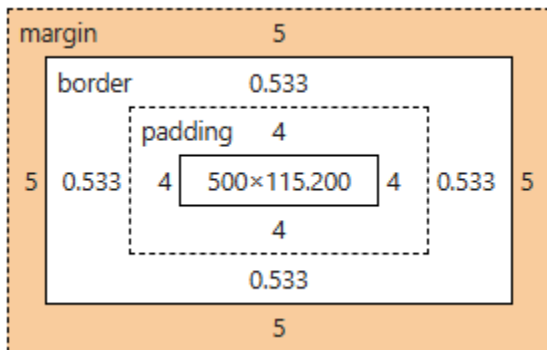
</head>
<body>
  <div class="c-box">
    content box = width(500px) + padding(4+4) + border(0.5 +0.5)
    <br>
    sum of all 3 will be greater then 500px
    <br>
    Layout may break if there is not available space
  </div>

  <div class="b-box">
    border box = width( < 500px ) + padding(4+4) + border(0.5 +0.5)
    sum of all 3 will be equal to 500px
    Layout will not break
  </div>

</body>
</html>

```

Box-sizing : content-box



Box-sizing : border-box

